SEGMENT TREE

```
class SegmentTree{
    public:
    int *tree;
    int size;
    int s_size=0;
    int *propagationArr;
    int left(int node){
        return ((node+1)*2-1);
    }
    int right(int node){
        return ((node+1)*2);
    }
    int mid(int begin,int end){
        return (((begin+1)+(end+1))/2)-1;
    }
    int minimum(int a,int b){
      if(a<b){
        return a;
      }
      return b;
    }
    void init(int node,int begin,int end,int arr[]){
        if(begin==end){
            s_size++;
            tree[node]=arr[begin];
            return;
        }
        init(left(node),begin,mid(begin,end),arr);
        init(right(node),mid(begin,end)+1,end,arr);
```

```
    tree[node]=minimum(tree[left(node)],tree[right(node)]);
    s_size++;
}
SegmentTree(int n,int arr[]){
    tree=new int[4*n];
    propagationArr=new int[4*n];
    for(int i=0;i<4*n;i++){
        propagationArr[i]=0;
    }
    size=4*n;
    init(0,0,n-1,arr);
}
void printTree(){
    for(int i=0;i<s_size;i++){
        cout << tree[i] << " ";
    }
    cout << endl;
    for(int i=0;i<s_size;i++){
        cout << propagationArr[i] << " ";
    }
    cout << endl;
}
int query_min(int node,int b,int e,int i,int j){
    if(propagationArr[node]!=0){
        tree[node]+=propagationArr[node];///tree[node]+=(propagationArr[node]*(e-b+1));
        if(b!=e){
            propagationArr[left(node)]+=propagationArr[node];
            propagationArr[right(node)]+=propagationArr[node];
        }
        propagationArr[node]=0;
```

```
    }
    if(i>e || j<b){
        return INT_MAX;
    }
    if(b>=i && e<=j){
        return tree[node];
    }
    int p1=query_min(left(node),b,mid(b,e),i,j);
    int p2 = query_min(right(node), mid(b,e)+1, e, i, j);
    return minimum(p1,p2);
}
void lazy_update(int node,int b,int e,int i,int j,int newVal){
    if(propagationArr[node]!=0){
        tree[node]+=propagationArr[node];///tree[node]+=(propagationArr[node]*(e-b+1));
        if(b!=e){
            propagationArr[left(node)]+=propagationArr[node];
            propagationArr[right(node)]+=propagationArr[node];
        }
        propagationArr[node]=0;
    }
    if(i>e || j<b){
        return;
    }
    if(i<=b && j>=e){
        tree[node]+=newVal;///tree[node]+=(newVal*(e-b+1));
        if(b!=e){
            propagationArr[left(node)]+=newVal;
            propagationArr[right(node)]+=newVal;
        }
        return;
```

```
        }
        lazy_update(left(node),b,mid(b,e),i,j,newVal);
        lazy_update(right(node),mid(b,e)+1,e,i,j,newVal);
        tree[node]=minimum(tree[left(node)],tree[right(node)]);
    }
};
```

🛠️ How to Build Prefix Sums

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> arr = {2, 3, 1, 5, 4};
    vector<int> prefix(arr.size());

    // Build prefix sum array
    prefix[0] = arr[0];
    for (int i = 1; i < arr.size(); i++) {
        prefix[i] = prefix[i-1] + arr[i];
    }

    // Original: [2, 3, 1, 5, 4]
    // Prefix:   [2, 5, 6, 11, 15]
```

```
    return 0;
}
```

📝 General Formula

For range [L, R] (0-based indexing):

text

```
sum(L, R) = prefix[R] - prefix[L-1]
```

Special case: When L = 0:

text

```
sum(0, R) = prefix[R]
```

```
struct charInfo{
    int arr[26]={0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};
};
struct ans{
    int highest=0;
    char c='\0';
};
class prefixSumChar{
    public:
    int size;
    charInfo *str;
    prefixSumChar(string s){
        size=s.size()+1;
        str=new charInfo[s.size()+1];
        charInfo count;
        int j=0;
        for(int i=1;i<=s.size();i++){
            count.arr[s[j]-'a']++;
            str[i]=count;
            j++;
        }
```

```
    }
    void print(){
        for(int i=0;i<size;i++){


            for(int j=0;j<26;j++){
                char c='a'+j;
                cout << c <<" " << str[i].arr[j] << " ";
            }
            cout << endl;
        }
    }
    ans mostFrequent(int r1,int r2){
        ans a;
        for(int i=25;i>=0;i--){
          if(a.highest<(str[r2].arr[i]-str[r1-1].arr[i])){
                a.highest=str[r2].arr[i]-str[r1-1].arr[i];
                a.c='a'+i;
          }
        }
        return a;
    }
};
```

Sparse table:

```
class sparsetable{///0 based indexing
    public:
    int** sparseTable;
    int s;
    sparsetable(int size,int arr[]){
        s=size;
        sparseTable=new int*[size];
        for(int i=0;i<size;i++){
            sparseTable[i]=new int[size];
        }
        for(int i=0;i<size;i++){
            sparseTable[i][0]=arr[i];
        }
        for(int j=1;j<=log2(size);j++){
            for(int i=0;i+(1<<j)-1<size;i++){
G            sparseTable[i][j]=__gcd(sparseTable[i][j-1],sparseTable[i+(1<<(j-1))][j-1]);
            }
        }
    }
    void rangesToStore(){
        for(int i=0;i<s;i++){
            for(int j=0;i+(1<<j)-1<s;j++){
                cout << "(" << i << "," << (i+(1<<j)-1) << ")" << "  ";
            }
            cout << endl;
        }
    }
    int query(int L,int R){
        int window=R-L+1;
```

```
        int j=log2(window);
        int left1=L;
        int right1=L+(1<<j)-1;
        int left2=R-((1<<j)-1);
        int right2=R;



        return __gcd(sparseTable[L][j],sparseTable[R-(1<<j)+1][j]);
    }
    void printStatus(){
        for(int i=0;i<s;i++){
            for(int j=0;(i+(1<<j)-1)<s;j++){
                cout << sparseTable[i][j] << " ";
            }
            cout << endl;
        }
    }
};
```

ARK [HASHING]

```
#include<iostream>
using namespace std;

int counter = 0;

void linear_prob(int ar[], int size, int pos, int ins){

    int ct = 0;
```

```
    for(int i =0; i<size; i++){

        ct++;

        if(ct==7){
            cout << "Input Abandoned.\n";
            break;
        }

        if(ar[pos+i]!=0){
            //cout << ar[pos+i] << endl ;
            cout << "Collision : Index " << pos+i << endl ;
        }
        else{
            counter++;
            float lf = float(counter) / float(size);
            ar[pos+i] = ins;
            cout << "Inserted : Index " << pos+i << "(L.F ="  << lf << ")" << endl;
            break;
        }
    }

}

void quadratic_pro(int ar[], int size, int pos, int ins){

    int ct = 0;

    for(int i =0; i<size; i++){
```

```
        ct++;

        if(ct==7){
            cout << "Input Abandoned.\n";
            break;
        }

        int nps = (pos+ (i*i))%size;

        if(ar[nps]!=0){
            //cout << ar[nps] << endl ;
            cout << "Collision : Index " << nps << endl ;
        }
        else{
            counter++;
            float lf = float(counter) / float(size);
            ar[nps] = ins;
            cout << "Inserted : Index " << nps << "(L.F ="  << lf << ")" << endl;
            break;
        }
    }
}

void double_hashing( int ar[], int size, int pos, int ins){
    int ct = 0;
    for(int i=0; i<size; i++){
        ct++;
        if(ct==7){
            cout << "Input Abandoned.\n";
```

```
            break;
        }
        int hash2 = (pos + (i*(7 - (ins%7))))%size;

        if(ar[hash2]!=0){
            cout << "Collision : Index " << hash2 << endl ;
        }
        else{
            counter++;
            float lf = float(counter) / float(size);
            ar[hash2] = ins;
            cout << "Inserted : Index " << pos+i << "(L.F ="  << lf << ")" << endl;
            break;
        }
    }

}
```

FAIYAZ [Hashing]

```
struct hashTable{
```

```
    bool isFull=false;
    int data;
};
class LinearProbing{
    public:
    int size;
    int element=0;
    LinearProbing(int s){
        size=s;
    }
    int hash(int data,int i){
        return (data+i)%size;
    }
    void linearProbing(int data,hashTable h[]){
        int i=0;
        while(h[hash(data,i)].isFull && i<6){
            cout << "Collision: Index-" <<hash(data,i) << endl;
            i++;
        }
        if(i<6){
            h[hash(data,i)].data=data;
            h[hash(data,i)].isFull=true;
            cout << "Inserted: Index-" << hash(data,i);
            element++;
            cout <<"(L.F= " <<float(element/(size*1.0))<<")" << endl;
        }
        else{
            cout << "Input Abandoned" << endl;
        }
    }
```

```
};
class QuadratingProbing{
    public:
    int size;
    int element=0;
    QuadratingProbing(int s){
        size=s;
    }
    int hash(int data,int i){
        return (data+i*i)%size;
    }
    void quardaticProbing(int data,hashTable h[]){
        int i=0;
        while(h[hash(data,i)].isFull && i<6){
            cout << "Collision: Index-" <<hash(data,i) << endl;
            i++;
        }
        if(i<6){
            h[hash(data,i)].data=data;
            h[hash(data,i)].isFull=true;
            cout << "Inserted: Index-" << hash(data,i);
            element++;
            cout <<"(L.F= " <<float(element/(size*1.0))<<")" << endl;
        }
        else{
            cout << "Input Abandoned" << endl;
        }
    }
```

```
};
class DoubleHashing{
    public:
    int size;
    int element=0;
    DoubleHashing(int s){
        size=s;
    }
    int hash2(int data){
        return (7)-data%(7);
    }
    int hash(int data,int i){
        return (data+i*hash2(data))%size;
    }
    void doubleHashing(int data,hashTable h[]){
        int i=0;
        while(h[hash(data,i)].isFull && i<6){
            cout << "Collision: Index-" <<hash(data,i) << endl;
            i++;
        }
        if(i<6){
            h[hash(data,i)].data=data;
            h[hash(data,i)].isFull=true;
            cout << "Inserted: Index-" << hash(data,i);
            element++;
            cout <<"(L.F= " <<float(element/(size*1.0))<<")" << endl;
        }
        else{
            cout << "Input Abandoned" << endl;
        }
```

```
    }
};



class FenwickTree{////0 based indexing
    int* tree;
    int length;
    public:
    int LSB(int number){
        return (number & (-number));
    }
    FenwickTree(int arr[],int size){
        tree=new int[size];
        length=size;
        for(int i=0;i<size;i++){
            tree[i]=arr[i];
        }
        for(int i=0;i<size;i++){
           int k=LSB(i+1);
           if(i+k<size){tree[i+k]+=tree[i];}
        }
    }
    void printTreeStatus(){
        for(int i=0;i<length;i++){
            cout << i+1 << ":" <<tree[i] << " ";
        }
        cout << endl;
```

```
    }
    int prefixSum(int index){
        int sum=0;
        int i=index;
        while(i>=0){
            sum+=tree[i];
            i-=LSB(i+1);
        }
        return sum;
    }
    int Query(int r1,int r2){
        int ans=prefixSum(r2)-prefixSum(r1-1);
        return ans;
    }
    void pointUpdate(int i,int x){
        while(i<length){
            tree[i]=tree[i]+x;
            i=i+LSB(i+1);
        }
    }  };
```

DSU (ARK)

```
void initialise(int n, int ar[], int rankr[]){
    for(int i=0; i<n; i++){
        ar[i] = i;
        rankr[i] = 0;
    }
}

void display(int n, int ar[], int rankr[]){
    for(int i=0; i<n; i++){
```

```
        cout << ar[i] << "(" << rankr[i] << ')' << " ";
    }
}

vector<int>func_calls;
int cntr = 0;

int find(int ar[], int it, int mode){
    if(mode==2){
        func_calls.push_back(it);
        cntr++;
    }
    if(ar[it]==it){
        return it;
    }
    return ar[it] = find(ar,ar[it],mode);
}

void unin(int i, int j, int ar[], int rankr[]){
    int repi = find(ar,i,3);
    int repj = find(ar,j,3);
    if(repi!=repj){
        if(rankr[repi]>rankr[repj]){
            ar[repj] = repi;
        }
        else if(rankr[repi] < rankr[repj]){
            ar[repi] = repj;
        }
        else{
            ar[repi] = repj;
```

```
            rankr[repj]++;
        }
    }
}

int main(){
    int n;
    cin >> n;
    int ar[n];
    int rankr[n];
    while(1){
        int cnt;
        cin >> cnt;
        if(cnt==1){
            initialise(n,ar,rankr);
            display(n,ar,rankr);
        }
        else if(cnt==2){
            int fnd;
            cin >> fnd;
            cout << find(ar,fnd,2) << endl;
            for(int i=0; i<cntr;i++){
                cout << "f(" << func_calls[i] << ')' << " ";
            }
            func_calls.clear();
            cntr = 0;
        }
        else if(cnt==3){
            int i,j;
            cin >> i >> j;
```

```
            unin(i,j,ar,rankr);
            display(n,ar,rankr);
        }
        else if(cnt==4){
            display(n,ar,rankr);
        }
        else{
            break;
        }
        cout << endl;
    }}
```

Disjoint Set [Faiyaz Version]

```
struct node{
    node* representative;
    int data;
    int rank;
};
class DSU{
    public:
    vector<node*> v;
    node* createNode(int i){
        node* newNode=new node();
        node* n1=createNode(num1);
        node* n2=createNode(num2);
        if(list[num1-1]->attached==NULL){
            list[num1-1]->attached=n2;
        newNode->data=i;
```

```
    newNode->representative=newNode;
    newNode->rank=0;
    return newNode;
}
void initialization(int n){
    for(int i=0;i<n;i++){
        node* newNode=createNode(i);
        v.push_back(newNode);
    }
}
node* find(node* n,bool willPrint){
    if(willPrint){
        cout << "f(" << n->data << ") ";
    }
    if(n->representative==n){
        return n;
    }
    return n->representative=find(n->representative,willPrint);
}
void Union(node* n1,node* n2){
    node* r1=find(n1,false);
    node* r2=find(n2,false);
    if(r1->rank<=r2->rank){
        r1->representative=r2;
    }
    else{
        r2->representative=r1;



    }
```

```
        if(r1->rank==r2->rank){
            r2->rank++;
        }
    }
    void print(){
        for(int i=0;i<v.size();i++){
            cout <<v[i]->representative->data << "(" <<v[i]->rank << ") ";
        }
        cout << endl;
    }
};
```

Adjacency List and Adjacency Matrix:(faiyaz)////undirected

```
struct node{
    node* attached;
    int data;
    node* tail;
};
class AdjacencyList{///////////1 based indexing!!!!!!!!!
    public:
    vector<node*> list;
    node* createNode(int data){
        node* newNode=new node();
        newNode->data=data;
```

```
    newNode->attached=NULL;
    newNode->tail=NULL;
    return newNode;
}
AdjacencyList(int V){
    for(int i=1;i<=V;i++){
        node* newNode=createNode(i);
        list.push_back(newNode);
    }
}
void insert(int num1,int num2){
    node* n1=createNode(num1);
    node* n2=createNode(num2);
    if(list[num1-1]->attached==NULL){
        list[num1-1]->attached=n2;
        list[num1-1]->tail=n2;
    }
    else{
        list[num1-1]->tail->attached=n2;
        list[num1-1]->tail=n2;
    }
    if(list[num2-1]->attached==NULL){
        list[num2-1]->attached=n1;
        list[num2-1]->tail=n1;
    }
    else{
        list[num2-1]->tail->attached=n1;
        list[num2-1]->tail=n1;
    }
}
```

```
    void printList(){
        for(int i=0;i<list.size();i++){
            cout << list[i]->data << ":";
            node* temp=list[i]->attached;
            while (temp!=NULL)
            {
                cout << temp->data << " ";
                temp=temp->attached;
            }
            cout << endl;
        }
    }
};
struct item{
    int data=0;
};
class AdjacencyMatrix{
    public:
    item** matrix;
    AdjacencyMatrix(int v){
        matrix=new item*[v];
        for(int i=0;i<v;i++){
            matrix[i]=new item[v];
        }
    }
    void insert(int num1,int num2){
        matrix[num1-1][num2-1].data=1;
        matrix[num2-1][num1-1].data=1;
    }
    void printMatrix(int v){
```

```
        for(int i=0;i<v;i++){
            for(int j=0;j<v;j++){
                cout << matrix[i][j].data << " ";
            }
            cout << endl;
        }
    }
};
```

Adjacency List-directed and topological sort(Faiyaz)

```
struct node{
    node* attached;
    int data;
    node* tail;
};
class AdjacencyListDirected{////////0 based indexing!!!!!!
    public:
    vector<node*> list;
    vector<int> inDegree;
    node* createNode(int data){
        node* newNode=new node();
        newNode->data=data;
        newNode->attached=NULL;
        newNode->tail=NULL;
        return newNode;
    }
    AdjacencyListDirected(int V){
        for(int i=0;i<V;i++){
            node* newNode=createNode(i);
            list.push_back(newNode);
            inDegree.push_back(0);
```

```
    }
}
void insert(int num1,int num2){
    node* n1=createNode(num1);
    node* n2=createNode(num2);
    if(list[num1]->attached==NULL){
        list[num1]->attached=n2;
        list[num1]->tail=n2;
    }
    else{
        list[num1]->tail->attached=n2;
        list[num1]->tail=n2;
    }
}
void printList(){
    for(int i=0;i<list.size();i++){
        cout << list[i]->data << ":";
        node* temp=list[i]->attached;
        while (temp!=NULL)
        {
            cout << temp->data << " ";
            temp=temp->attached;
        }
        cout << endl;
    }
}
void createIndegree(){
    for(int i=0;i<list.size();i++){
        node* temp=list[i]->attached;
        while (temp!=NULL)
```

```
        {
            inDegree[temp->data]++;
            temp=temp->attached;
        }
    }
}
bool topologicalSort(int arr[],int v){
    createIndegree();
    vector<int> temp;
    temp=inDegree;
    queue<int> q;
    for(int i=0;i<temp.size();i++){
        if(temp[i]==0){
            q.push(i);
        }
    }
    int arr_index=0;
    while(!q.empty()){
        int index=q.front();
        q.pop();
        node* tNode=list[index]->attached;
        while(tNode!=NULL){
            temp[tNode->data]--;
            if(temp[tNode->data]==0){
                q.push(tNode->data);
            }
            tNode=tNode->attached;
        }
        arr[arr_index]=index;
        arr_index++;
```

```
        }
        for(int i=0;i<temp.size();i++){
            if(temp[i]!=0){
                return false;
            }
        }
        return true;
    }
};
```

BFS—-Faiyaz

```
struct node{
    node* attached;
    int data;
    node* tail;
    string color="White";
    int distFromSource=-1;
    node* pi=NULL;
};
class AdjacencyList{///////////1 based indexing!!!!!!!!!
    public:
    vector<node*> list;
    vector<node*> pathToSource;
    node* createNode(int data){
```

```
    node* newNode=new node();
    newNode->data=data;
    newNode->attached=NULL;
    newNode->tail=NULL;
    return newNode;
}
AdjacencyList(int V){
    for(int i=1;i<=V;i++){
        node* newNode=createNode(i);
        list.push_back(newNode);
        pathToSource.push_back(newNode);
    }
}
void insert(int num1,int num2){
    node* n1=createNode(num1);
    node* n2=createNode(num2);
    if(list[num1-1]->attached==NULL){
        list[num1-1]->attached=n2;
        list[num1-1]->tail=n2;
    }
    else{
        list[num1-1]->tail->attached=n2;
        list[num1-1]->tail=n2;
    }
    if(list[num2-1]->attached==NULL){
        list[num2-1]->attached=n1;
        list[num2-1]->tail=n1;
    }
    else{
        list[num2-1]->tail->attached=n1;
```

```
        list[num2-1]->tail=n1;
    }
}
void printList(){
    for(int i=0;i<list.size();i++){
        cout << list[i]->data << ":";
        node* temp=list[i]->attached;
        while (temp!=NULL)
        {
            cout << temp->data << " ";
            temp=temp->attached;
        }
        cout << endl;
    }
}
vector<int> BFS(int s){
    vector<int>  ans;
    queue<node*> q;
    node* source=NULL;
    for(int i=0;i<list.size();i++){
        if(list[i]->data!=s){
            list[i]->color="WHITE";
            list[i]->distFromSource=-1;
            list[i]->pi=NULL;
        }
        else{
            source=list[i];
        }
    }
    source->color="GRAY";
```

```
    source->distFromSource=0;
    source->pi=NULL;
    q.push(source);
    while(!q.empty()){
        node* u=q.front();
        q.pop();
        node* temp=list[u->data-1]->attached;
        while(temp!=NULL){
            if(list[temp->data-1]->color=="WHITE"){
                list[temp->data-1]->color="GRAY";
                list[temp->data-1]->distFromSource=u->distFromSource+1;
                list[temp->data-1]->pi=u;
                q.push(list[temp->data-1]);
            }
            temp=temp->attached;
        }
        u->color="BLACK";
        ans.push_back(u->data);
    }
    return ans;
}
void distancePrint(){
    for(int i=0;i<list.size();i++){
        cout << list[i]->data << "(" << list[i]->distFromSource << ") ";
    }
    cout << endl;
}
void pathFromSource(int s){
    for(int i=0;i<list.size();i++){
        vector<int> path;
```

```
            node* temp=list[i];
            while(temp!=NULL){
                path.push_back(temp->data);
                if(temp->pi==NULL){
                    break;
                }
                temp=list[temp->pi->data-1];
            }
            cout << list[i]->data << ":";
            for(int j=path.size()-1;j>=0;j--){
                cout << path[j];
                if(j!=0){
                    cout << "->";
                }
            }
            cout << endl;
        }
    }
    void edgesPrint(){
        for(int i=0;i<list.size();i++){
            if(list[i]->pi!=NULL){
                cout << list[i]->data << " " << list[i]->pi->data << endl;
            }
        }
    }
};
```

DFS—-Faiyaz

```
struct node{
    node* attached;
    int data;
    node* tail;
    string color;
    node* pi;
    int start_time;
    int end_time;
};
class AdjacencyListDirected{////////0 based indexing!!!!!!
    public:
    vector<node*> list;
    vector<int> inDegree;
    vector<pair<int,int>> treeEdgesDFS;
    node* createNode(int data){
        node* newNode=new node();
        newNode->data=data;
        newNode->attached=NULL;
        newNode->tail=NULL;
        newNode->color="WHITE";
        newNode->pi=NULL;
        newNode->start_time=0;
        newNode->end_time=0;
        return newNode;
    }
    AdjacencyListDirected(int V){
        for(int i=0;i<V;i++){
            node* newNode=createNode(i);
            list.push_back(newNode);
            inDegree.push_back(0);
```

```
    }
}
void insert(int num1,int num2){
    node* n1=createNode(num1);
    node* n2=createNode(num2);
    if(list[num1]->attached==NULL){
        list[num1]->attached=n2;
        list[num1]->tail=n2;
    }
    else{
        list[num1]->tail->attached=n2;
        list[num1]->tail=n2;
    }
}
void printList(){
    for(int i=0;i<list.size();i++){
        cout << list[i]->data+1 << ":";
        node* temp=list[i]->attached;
        while (temp!=NULL)
        {
            cout << temp->data+1 << " ";
            temp=temp->attached;
        }
        cout << endl;
    }
}
vector<int> DFS(){
    vector<int> dfs_seq;
    treeEdgesDFS.clear();
    for(int i=0;i<list.size();i++){
```

```
        list[i]->color="WHITE";
        list[i]->pi=NULL;
    }
    int time=0;
    for(int i=0;i<list.size();i++){
        if(list[i]->color=="WHITE"){
            DFS_Visit(list[i],time,dfs_seq);
        }
    }
    return dfs_seq;
}

void DFS_Visit(node* n,int &time,vector<int> &dfs_seq){
    dfs_seq.push_back(n->data);
    time++;
    n->start_time=time;
    n->color="GRAY";
    node* temp=n->attached;
    while(temp!=NULL){
        if(list[temp->data]->color=="WHITE"){
            treeEdgesDFS.push_back({n->data,temp->data});
            list[temp->data]->pi=n;
            DFS_Visit(list[temp->data],time,dfs_seq);
        }
        temp=temp->attached;
    }
    n->color="BLACK";
    time++;
    n->end_time=time;
}
```

```
void showTimeStamp(){
    for(int i=0;i<list.size();i++){
        cout << list[i]->data+1 << "("<< list[i]->start_time << "/" << list[i]->end_time << ")" << endl;
    }
}
string edgeTypeDetermine(vector<pair<int,int>> &treeEdgesSorted,
                        pair<int,int> node1Time,
                        pair<int,int> node2Time,
                        int node1Data,
                        int node2Data){
    if(node1Time.first<node2Time.first && node1Time.second>node2Time.second){
        pair<int,int> edge;
        edge.first=node1Data;
        edge.second=node2Data;
        auto a=find(treeEdgesDFS.begin(),treeEdgesDFS.end(),edge);
        if(a==treeEdgesSorted.end()){
            return "Forward Edge";
        }
        else{
            return "Tree Edge";
        }
    }
    else if((node1Time.first>node2Time.first && node1Time.second<node2Time.second) || node1Data==node2Data){
        return "Back Edge";
    }
    else{
        return "Cross Edge";
    }
}
```

```
void printEdgeType(){
    for(int i=0;i<list.size();i++){
        node* temp=list[i]->attached;
        sort(treeEdgesDFS.begin(),treeEdgesDFS.end());
        while (temp!=NULL)
        {
            pair<int,int> node1Time={list[i]->start_time,list[i]->end_time};
            pair<int,int> node2Time={list[temp->data]->start_time,list[temp->data]->end_time};
```

```
            cout << list[i]->data+1 << " " << temp->data+1 <<":" <<
edgeTypeDetermine(treeEdgesDFS,node1Time,node2Time,list[i]->data,temp->data)<< endl;
            temp=temp->attached;
        }
    }
  }
};
```